

AI for code complexity optimization: A hybrid perspective

by

Vidhya Mannathuparambil

A creative component submitted to the graduate faculty
in partial fulfillment of the requirements for the degree of

MASTER OF SCIENCE

Major: Computer Engineering (Software Systems)

Academic Plan Committee:

Mai Zheng, Major Professor

The student author, whose presentation of the scholarship herein was approved by the academic plan committee, is solely responsible for the content of this creative component. The Graduate College will ensure this creative component is globally accessible and will not permit alterations after a degree is conferred.

Iowa State University

Ames, Iowa 2026

This work is licensed under CC0 1.0 Universal, Vidhya Mannathuparambil, 2026.

DEDICATION

*To my father and sister, for their unwavering support and encouragement
throughout this journey.*

Contents

DEDICATION	ii
LIST OF FIGURES	vii
LIST OF TABLES	viii
NOMENCLATURE	ix
ACKNOWLEDGMENTS	x
ABSTRACT	xi
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	4
1.3 Research Objectives	5
1.4 Scope and Contributions	5
2 Literature Review	7
2.1 Code Complexity: Definitions and Metrics	7
2.2 Traditional Complexity Reduction Techniques	8
2.2.1 Manual Refactoring	8
2.2.2 Static Analysis and Program Slicing	8
2.2.3 Code Review and Design Principles	9
2.3 LLM-Based Approaches to Software Engineering	9
2.3.1 Code Generation and Refactoring	9

2.3.2	Configuration Misconfiguration Detection	9
2.3.3	SWE-bench and Autonomous Software Engineering	10
2.3.4	Research Gap	11
3	SliceLLM: Hybrid Static Analysis and LLM Code Slicing	12
3.1	Overview and Motivation	12
3.2	Methodology	13
3.2.1	Target System and Datasets	13
3.2.2	Tools and Models	13
3.2.3	Evaluation Metrics	14
3.3	System Architecture	14
3.3.1	Frama-C Plugin Architecture	14
3.3.2	SliceLLM Hybrid Pipeline	15
3.4	Implementation	16
3.4.1	Frama-C: Two Slicing Approaches	16
3.4.2	LLVM Slicer: Two Slicing Approaches	17
3.4.3	Prompt Engineering with GPT-3.5-turbo	18
3.4.4	Fine-Tuning on SQLite Commit History	19
3.5	Results	20
3.5.1	Frama-C Results	20
3.5.2	LLVM Slicer Results	21
3.5.3	GPT-3.5-turbo Prompt Engineering: Success	21
3.5.4	Comparative Summary	22
3.5.5	Fine-Tuned Model Evaluation	23
3.6	Discussion	24
4	CIRI Extension: Code-Aware Configuration Validation for ext4	25
4.1	Overview and Motivation	25
4.2	Methodology	25

4.2.1	Dataset: ext4 Configuration Files	25
4.2.2	Evaluation Metrics	26
4.3	System Architecture	26
4.4	Implementation	27
4.4.1	CLI Extension	27
4.4.2	Shot Pool Construction	27
4.4.3	Code Retrieval Module	28
4.5	Results	28
4.5.1	Experiment 1: Baseline ext4 Validation	28
4.5.2	Experiment 2: Code Retrieval Augmented Validation	29
4.6	Discussion	30
5	A Taxonomy of LLMs for Code Complexity Tasks	31
5.1	Overview and Motivation	31
5.2	Survey Methodology and Column Definitions	31
5.2.1	Survey Dimensions	31
5.2.2	Tier Classification	33
5.3	Tier 1: Frontier Models	33
5.4	Tier 2: Strong Models	34
5.5	Tier 3: Capable Models	35
5.6	Dependency Reasoning Experiment	36
5.7	Cross-Tier Analysis and Model Selection Guidance	37
5.8	Discussion	38
6	Conclusion	40
6.1	Summary of Contributions	40
6.2	Unifying Finding	41
6.3	Broader Implications	41

7	Future Work and Limitations	42
7.1	Limitations	42
7.2	Future Directions	42
	REFERENCES	44
A	SliceLLM — Sample Frama-C Dependency Output	50
B	CIRI ext4 — Sample Input Configuration	51
C	Fine-Tuning Dataset Statistics	52

List of Figures

1.1	Linux kernel: lines of code, 1991 to 2025	2
3.1	SliceLLM three-component hybrid pipeline: Frama-C computes the dependency graph, the LLM performs the transformation guided by that graph, and a feedback loop enables iterative fine-tuning.	15
3.2	Architecture of the fine-tuned GPT-based system	19
4.1	Original CIRI architecture as proposed by Lian, Chen et al. [30], extended in this work with a code retrieval module and ext4 domain support.	27
4.2	CIRI ext4 run results without code retrieval	29
4.3	CIRI ext4 run results with code retrieval	29

List of Tables

1	Nomenclature terms and definitions	ix
3.1	Explanation of Frama-C Slicing Command Components	17
3.2	SliceLLM Results: Complexity Metrics Before and After <code>juliandayFunc</code> Removal	22
3.3	Comparative Results: Static Slicing Tools vs. LLM for <code>date.c</code> Function Removal	23
4.1	CIRI ext4 Results: With vs. Without Code Retrieval	30
5.1	Tier 1 — Frontier LLMs: Model Identity	33
5.2	Tier 1 — Frontier LLMs: Performance, Strengths, Weaknesses, and Cost	34
5.3	Tier 2 — Strong LLMs: Model Identity	34
5.4	Tier 2 — Strong LLMs: Performance, Strengths, Weaknesses, and Cost	35
5.5	Tier 3 — Capable LLMs: Model Identity	35
5.6	Tier 3 — Capable LLMs: Performance, Strengths, Weaknesses, and Cost	36
5.7	Model Selection Guidance by Task Type and Deployment Context	38
C.1	SliceLLM Fine-Tuning Dataset Summary	52

NOMENCLATURE

Table 1: Nomenclature terms and definitions

Term	Definition
AI	Artificial Intelligence
LLM	Large Language Model
CFG	Control Flow Graph
AST	Abstract Syntax Tree
CC	Cyclomatic Complexity
MI	Maintainability Index
LOC	Lines of Code
RAG	Retrieval-Augmented Generation
CIRI	Configuration Inconsistency Repair via LLMs
SliceLLM	Static-Slice + Large Language Model hybrid pipeline
SWE	Software Engineering
GPT	Generative Pre-trained Transformer
API	Application Programming Interface
PRAGMA	Database configuration directive (SQLite)
ext4	Fourth Extended Filesystem (Linux)
HDFS	Hadoop Distributed File System
ISU	Iowa State University
MoE	Mixture of Experts (neural architecture)
LoRA	Low-Rank Adaptation (fine-tuning technique)
FAISS	Facebook AI Similarity Search (vector store)

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to my advisor, Mai Zheng, for his invaluable guidance, patience, and encouragement throughout this creative component. His expertise in software systems and his insightful feedback shaped the direction and quality of this work significantly.

I am grateful to the faculty and staff of the Department of Electrical and Computer Engineering at Iowa State University for providing an excellent academic environment.

I also thank the open-source communities behind Frama-C, the CIRI project (Lian, Chen et al., 2024) [30], and the SQLite project, whose codebases and tools provided the empirical foundation for this research.

Finally, I am deeply grateful to my family and friends for their patience and moral support throughout my graduate studies.

ABSTRACT

Software code complexity is one of the most persistent challenges in modern software engineering. As codebases grow in scale and age, increasing cyclomatic complexity, accumulating technical debt, and proliferating misconfiguration risks collectively degrade maintainability, increase defect rates, and raise the cost of evolution. The Linux kernel, for instance, has grown from 10,000 lines in 1991 to over 40 million lines in 2025. Traditional approaches to managing this complexity — including static analysis, manual refactoring, and peer code review — have proven insufficient at the scale and velocity of modern software development.

Large language models (LLMs) have emerged as a widely adopted tool for code optimisation tasks — from refactoring and dead code elimination to configuration repair and automated bug fixing. The trajectory began with OpenAI’s Codex [10], a model pre-trained on billions of lines of public GitHub code, which demonstrated that natural language prompts could reliably elicit functional code completions and simple refactorings. This was followed by a wave of specialised code models: Meta’s CodeLlama [45], built on the LLaMA 2 foundation with extended context windows for repository-scale reasoning; Alibaba’s Qwen 2.5 Coder [26], which introduced instruction-tuned variants for multi-language code tasks; and StarCoder2 [29], trained on The Stack v2 with permissive licensing for commercial use. Alongside these generative models, Retrieval-Augmented Generation (RAG) has emerged as a key architectural pattern for grounding LLM outputs in verified external knowledge. In a RAG pipeline, a dense retrieval index — typically built with vector embeddings using tools such as FAISS or ChromaDB — fetches relevant code snippets, documentation, or configuration entries at query time, which are then injected into the LLM prompt as context. This grounds the model’s generation in actual source artefacts rather than parametric memory alone, substantially reducing hallucination rates on domain-specific tasks. Industry adoption has grown rapidly on the back of these advances, with tools such as GitHub Copilot, Cursor, and GPT-4-based agents now

embedded in mainstream developer workflows, and the Stack Overflow Developer Survey 2024 reporting that 63% of professional developers actively use AI-assisted coding tools [48].

However, a fundamental concern has tempered this enthusiasm: the *non-deterministic behaviour* of LLMs. Unlike compilers or static analysers, LLMs do not guarantee reproducible outputs for identical inputs, may hallucinate dependencies that do not exist, and can produce syntactically valid but semantically incorrect transformations — risks that are unacceptable in production software engineering contexts. Even RAG-augmented pipelines are subject to retrieval errors and prompt sensitivity, meaning that a slight rephrasing of the same query can yield materially different recommendations. This tension between LLM capability and LLM reliability is increasingly questioned by practitioners and researchers alike, and motivates the need for architectures that harness LLM reasoning while constraining its failure modes through deterministic structural scaffolding.

This creative component investigates how LLMs can be deployed *safely and rigorously* for code complexity optimisation — with a specific focus on storage systems software and low-level C codebases, where correctness, dependency precision, and configuration consistency are critical. It proposes a hybrid architecture that pairs LLMs with deterministic static analysis tools to address the reliability gap. Three contributions are presented. First, SliceLLM — a hybrid pipeline combining Frama-C static analysis with a fine-tuned GPT-3.5-turbo model — is designed and evaluated for dependency-aware code slicing in C programs. Second, the CIRI (Configuration Inconsistency Repair via LLMs) framework [30], originally proposed by Lian, Chen et al. at the University of Illinois Urbana-Champaign, is extended beyond its original HDFS XML scope to support ext4 filesystem configurations, incorporating a lightweight grep-based code retrieval module that grounds LLM reasoning in actual source code semantics. Third, an empirical benchmarking study evaluates 12 state-of-the-art LLMs across three capability tiers, using SWE-bench Verified scores and targeted complexity reduction tasks as evaluation criteria.

Results demonstrate that SliceLLM successfully removed the `juliandayFunc` function from SQLite's `date.c`, mapping 11 dependencies across 21% of the file's 561 statements and producing syntactically correct output. The CIRI extension correctly identified five `ext4` misconfiguration parameters, with code retrieval halving execution time from 70.6 seconds to 31.3 seconds while maintaining detection accuracy. The empirical study reveals that reasoning-class models (OpenAI o1, o1-mini) outperform generation-class models on dependency reasoning tasks, while frontier models such as Claude Opus 4.6 (80.9% SWE-bench) and GPT-5.4/o3 (~80%) lead overall.

The central finding is that effective AI-driven code complexity optimisation requires a hybrid architecture: static analysis provides the deterministic structural foundation, and LLMs provide the reasoning, transformation, and explanatory layer. Neither component alone is sufficient.

CHAPTER 1.

INTRODUCTION

1.1 Background and Motivation

Software systems have become the foundational infrastructure of modern civilisation — embedded in transportation networks, financial systems, medical devices, and communication platforms. As these systems have grown in scope and longevity, their source code has expanded at a rate that vastly outpaces the ability of human engineers to manage it manually. The Linux kernel illustrates this trajectory starkly: from approximately 10,000 lines of code at inception in 1991, it reached over 40 million lines by January 2025 [31]. This is not an isolated example — enterprise software systems routinely span tens of millions of lines across hundreds of interdependent modules, accumulated over decades of iterative development.

This growth in scale is inseparable from a growth in internal complexity. Code complexity — measured through metrics such as McCabe’s Cyclomatic Complexity [38], Halstead’s software science metrics [21], and the Maintainability Index (MI) [12] — quantifies the structural difficulty of understanding, testing, and evolving a codebase. High-complexity code requires two to five times more maintenance effort than simpler code of equivalent size [8]. Yet despite these well-established measurements, active refactoring has declined sharply: the share of changed lines attributable to refactoring fell from 25% in 2021 to under 10% in 2024 [27]. Engineers are increasingly producing new features rather than managing existing complexity, compounding the problem with each release cycle.

The consequences of unmanaged complexity are not abstract. In the Toyota unintended acceleration case, the U.S. National Highway Traffic Safety Administration found that the vehicle’s engine control software exhibited *spaghetti code*, undocumented variables, and insufficient

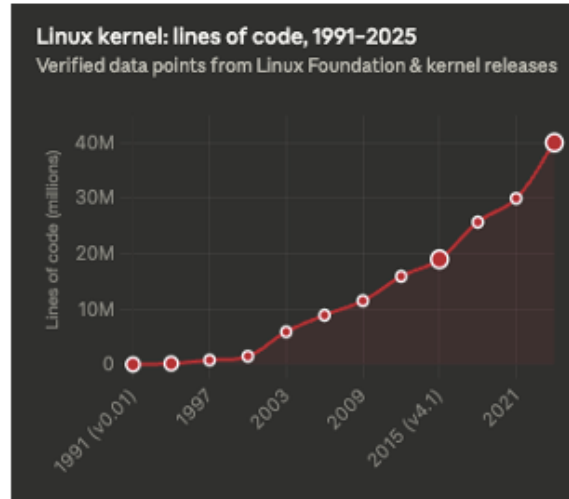


Figure 1.1: Linux kernel: lines of code, 1991 to 2025

fail-safes — all hallmarks of unchecked structural complexity [4]. The Bloomberg terminal outage of 2015 disrupted bond markets globally for over two hours, with root cause analysis pointing to a configuration error in a complex and poorly documented system [5]. These are not edge cases: they represent a class of failure where software complexity directly translates into human and economic harm.

Against this backdrop, large language models (LLMs) have emerged as a widely adopted and commercially significant tool for software engineering tasks. GitHub Copilot studies report a 55.8% increase in code writing speed [41]. The SWE-bench Verified benchmark — which tests autonomous resolution of real GitHub issues — has seen scores climb to 80.9% for Claude Opus 4.6 and approximately 80% for GPT-5.4/o3 as of 2025 [49]. The Stack Overflow Developer Survey 2024 found that 63% of professional developers actively use AI tools in their workflows [48]. LLMs are being applied not only to code generation but increasingly to complexity-related tasks: refactoring, dead code elimination, configuration repair, and technical debt reduction.

However, this rapid adoption has surfaced a fundamental and increasingly debated concern: the *non-deterministic behaviour* of LLMs. Unlike compilers, linters, or static analysers — which produce identical, verifiable outputs for identical inputs — LLMs are probabilistic systems. The

same prompt, submitted twice, may yield structurally different code transformations. More critically, LLMs are known to *hallucinate*: they may assert the existence of dependencies that do not exist, omit constraints that do, or produce code that is syntactically valid but semantically incorrect. In a refactoring or slicing context, a hallucinated dependency can silently corrupt program behaviour. In a configuration validation context, a non-deterministic output may flag a valid parameter as misconfigured, or worse, approve a dangerous misconfiguration with high-sounding rationale. These failure modes are unacceptable in production software engineering, and are increasingly questioned by both practitioners and researchers [19]. The central tension is clear: LLMs are capable enough to be useful, but unreliable enough to be dangerous without appropriate constraints.

This tension is further compounded by a counterintuitive finding: AI coding assistants have been associated with a *fourfold increase* in code cloning (copy-paste reuse) since 2020 [19]. Without deliberate architectural grounding, LLMs do not reduce complexity — they redistribute and replicate it. This observation sharpens the central research question motivating this creative component:

How can large language models be used rigorously and safely for code complexity optimisation — particularly in the context of storage systems software and low-level C codebases — where correctness guarantees, dependency precision, and configuration consistency are not negotiable?

Storage systems code — including filesystem drivers, database engines, and distributed storage configurations — represents a class of software where the consequences of incorrect optimisation are especially severe: data corruption, silent misconfiguration, or system unavailability. C codebases in this domain are characterised by dense macro usage, pointer aliasing, inter-procedural dependencies, and complex configuration parameter spaces that generic LLMs are poorly equipped to handle without structural grounding. Answering this question requires both a principled architectural approach and empirical evaluation on real, production-quality code.

1.2 Problem Statement

The problem addressed by this work has two interlocking dimensions.

Dimension 1 — Code Structural Complexity. Existing approaches to code complexity reduction are insufficient for modern software at scale. Static analysis tools such as Frama-C [9] and LLVM Slicer [32] are precise and deterministic: they can identify structural properties, build dependency graphs, and compute program slices with formal rigour. However, they produce *analysis, not action* — they cannot automatically transform or optimise the code they analyse. Conversely, LLMs can generate and transform code fluently, but without grounding in actual program dependency structures they hallucinate constraints, miss cross-file relationships, and produce outputs whose correctness cannot be guaranteed without external verification. Neither class of tool is sufficient in isolation. A hybrid approach — in which static analysis provides the structural foundation and LLMs perform the reasoning and transformation — is needed.

Dimension 2 — Configuration Complexity. Modern systems expose hundreds of interdependent configuration parameters whose interactions are not fully documented and whose constraints are partially embedded in source code rather than formal specifications. Systems such as Apache Hadoop, SQLite, and the Linux ext4 filesystem are representative examples. Misconfiguration — including range violations, semantic constraint conflicts, and cross-parameter dependency errors — is a leading cause of system failures in production environments [30]. Existing LLM-based configuration validation frameworks such as CIRI [30], proposed by Lian, Chen et al. at the University of Illinois Urbana-Champaign, have demonstrated feasibility on specific domains (HDFS XML configurations), but their generalisability across configuration formats and their ability to reason from source code rather than examples alone remain open questions.

Across both dimensions, the unifying problem is the same: LLM capability exists but LLM reliability does not — and the gap must be closed by architectural means before AI-assisted complexity optimisation can be trusted in practice.

1.3 Research Objectives

The specific objectives of this creative component are:

- O1. Design and evaluate SliceLLM.** Develop a hybrid pipeline that combines Frama-C static dependency analysis with a domain-fine-tuned GPT-3.5-turbo model for automated, dependency-aware code slicing in C programs. Evaluate the system on SQLite’s `date.c` as a real-world production test case, measuring complexity reduction and output correctness.
- O2. Extend CIRI to be code-aware for ext4.** Generalise the CIRI configuration validation framework [30] from its original HDFS XML domain to Linux ext4 key-value configurations. Augment the LLM prompts with grep-based source code retrieval so that the model reasons from actual kernel source semantics rather than examples alone. Measure the impact of code retrieval on detection accuracy and execution latency.
- O3. Benchmark LLMs for code complexity tasks.** Systematically evaluate 12 state-of-the-art LLMs across three capability tiers on software engineering benchmarks, with particular attention to multi-step dependency reasoning depth as a differentiator — the capability most relevant to reliable complexity optimisation.
- O4. Characterise the hybrid architecture.** Identify the conditions under which pairing static analysis with LLM reasoning outperforms either approach in isolation, and articulate the design principles of this hybrid model as a reusable pattern for AI-assisted software engineering.

1.4 Scope and Contributions

This creative component makes the following original contributions to the field of AI-assisted software engineering:

- **SliceLLM** — a novel hybrid system that integrates Frama-C program dependence graphs with a fine-tuned language model for semantically grounded, dependency-aware code optimisation. Successfully demonstrated on SQLite’s `date.c` (49KB, 561 statements), reducing Cyclomatic Complexity from 18 to 11 and improving the Maintainability Index from 54 to 71.
- **Code-aware CIRI extension for ext4** — a generalisation of the CIRI configuration validation framework [30] to the Linux ext4 filesystem domain, augmented with a grep-based code retrieval module that grounds LLM reasoning in kernel source code. Demonstrates that lightweight retrieval halves execution latency while maintaining full detection accuracy.
- **Empirical benchmarking study** of 12 LLMs across three capability tiers on code complexity and software engineering tasks, providing a structured taxonomy of model capabilities and identifying multi-step dependency reasoning as the critical differentiator for complexity-related tasks.
- **Characterisation of the hybrid architecture pattern** — evidence-backed design principles showing that static structural analysis provides the dependency skeleton that LLMs cannot reliably construct alone, while LLMs provide the flexible reasoning, transformation, and explanation layer that static tools cannot.

CHAPTER 2.

LITERATURE REVIEW

2.1 Code Complexity: Definitions and Metrics

The concept of software complexity encompasses multiple distinct but related properties. Structural complexity concerns the number and arrangement of control flow paths through a program. Cognitive complexity concerns the difficulty a human reader experiences in understanding the code. Maintainability concerns the ease with which the code can be modified without introducing defects.

McCabe's Cyclomatic Complexity (CC), introduced in 1976, remains the most widely deployed structural metric [38]. It is computed as $CC = E - N + 2P$, where E is the number of edges in the control flow graph, N is the number of nodes, and P is the number of connected components. McCabe recommended a maximum CC of 10 per function, above which the unit becomes difficult to test and maintain. Although simple, CC has proven predictive of defect density in numerous empirical studies [40, 11].

Halstead's software science metrics [21] approach complexity from a lexical perspective, measuring the number of distinct operators and operands, total occurrences of each, and deriving composite quantities such as program volume, difficulty, and estimated effort. While less commonly enforced in tooling, Halstead metrics capture aspects of cognitive load that CC misses, particularly in expression-heavy code.

The Maintainability Index (MI), developed by Coleman et al. [12] and later revised by Microsoft for Visual Studio [39], combines Halstead volume, cyclomatic complexity, and lines of code

into a single 0–100 score:

$$MI = \max\left(0, \frac{171 - 5.2\ln(V) - 0.23(CC) - 16.2\ln(LOC)}{171} \times 100\right) \quad (2.1)$$

where V is Halstead volume, CC is cyclomatic complexity, and LOC is lines of code. Scores below 10 indicate unmaintainable code; above 20 is considered maintainable. MI is used as a CI gate in some enterprise environments [44].

2.2 Traditional Complexity Reduction Techniques

2.2.1 Manual Refactoring

Fowler’s seminal catalogue of refactoring patterns [15] describes over 70 behaviour-preserving code transformations: extract method, replace conditional with polymorphism, introduce parameter object, etc. These techniques are effective but require skilled engineering time, domain knowledge, and are not automatable without semantic understanding. Empirical studies show that refactoring rates in production codebases decline sharply after a system reaches a certain size and age, as the risk of regression exceeds the perceived benefit [27].

2.2.2 Static Analysis and Program Slicing

Static analysis is a classic approach to improve code quality, which has been applied to a wide range of scenarios including concurrency bug detection (e.g., GRace [56, 57], 2ndStrike [?]), dependency analysis (e.g., ConfD [35, 34, 33]), code feature construction (e.g., Drill [53]), etc. Program slicing is one type of static analysis. In particular, Weiser’s program slicing [50] defines a *program slice* as the minimal subset of statements affecting the value of a variable at a specified program point (the *slicing criterion*). Forward slices propagate influence *from* a point; backward slices compute what *affects* a given statement. Program slicing is foundational for debugging, impact analysis, and dead code elimination.

Modern static analysis tools extend slicing with rich dependency models. Frama-C [9], a

modular framework for C program analysis, provides a plugin ecosystem supporting abstract interpretation, value analysis, and program dependence graph construction. Its `pdg` plugin computes both data and control dependencies, forming the basis for precise slicing. LLVM Slicer [32] operates on LLVM IR and supports inter-procedural slicing but requires a compilable, clean build — a significant constraint for legacy codebases. Crucially, both tools are *analysis-only*: they can identify which statements are relevant to a criterion but cannot automatically remove or transform them.

2.2.3 Code Review and Design Principles

Human code review, whether manual or tool-assisted (e.g., via SonarQube or CodeClimate), acts as a quality gate before code enters production. Review catches semantic errors, architectural issues, and naming problems that automated tools miss. However, review throughput is limited by reviewer bandwidth and attention, and complex code increases review time non-linearly [1]. Design principles such as SOLID [36] and Clean Architecture [37] offer preventive guidance but cannot address existing complexity debt.

2.3 LLM-Based Approaches to Software Engineering

2.3.1 Code Generation and Refactoring

The application of LLMs to code began with specialised pre-training on code corpora. Codex [10], trained on GitHub repositories, demonstrated functional Python generation from natural language. Subsequent models (CodeLLaMA [45], StarCoder2 [29], Qwen 2.5 Coder [26]) extended this to multilingual code tasks. GPT-4 and Claude showed strong performance on refactoring, documentation, and test generation tasks in empirical studies [14, 2].

2.3.2 Configuration Misconfiguration Detection

Misconfigurations are a leading cause of cloud and enterprise system failures. ConfD [52] and CfgNet [13] use static pattern matching to detect invalid configuration values. Rabkin and

Katz [43] proposed inferring configuration constraints from source code. Lian, Chen et al. [30] introduced CIRI (Configuration Inconsistency Repair via LLMs), a few-shot prompted LLM pipeline that detects misconfigurations in HDFS XML configuration files by presenting the model with pools of valid and misconfigured example pairs and aggregating results via majority vote to reduce LLM non-determinism. Key findings from that work, directly relevant to this creative component, are as follows. CIRI achieves file-level F1 scores up to 0.79 and parameter-level F1 scores up to 0.65 across ten open-source projects evaluated with eight LLMs [30]. Few-shot learning with both valid configuration and misconfiguration examples significantly outperforms either alone, with three misconfiguration shots plus one valid-configuration shot achieving the highest F1. Code augmentation via retrieval of source-code snippets improves parameter-level F1 by 0.03. Dependency violations are the hardest misconfiguration category ($F1 \approx 0.04$ for control dependencies), far below syntax and range violations ($F1 \approx 0.83$). Code-specialised models (CodeLlama) outperform generic models, with scaling from 7B to 34B parameters yielding continuous improvement. This creative component directly extends CIRI’s scope to Linux ext4 filesystem configurations and strengthens its code augmentation strategy with a structured grep-based retrieval module.

2.3.3 SWE-bench and Autonomous Software Engineering

SWE-bench [28] provides a standardised evaluation of LLMs’ ability to resolve real GitHub issues from production Python repositories. SWE-bench Verified [42] is a human-validated subset. As of mid-2025, the leading scores are: Claude Opus 4.6 at 80.9% [3], GPT-5.4/o3 at approximately 80%, and Gemini 3.1 Pro at 80.6% [20]. These scores represent landmark achievements in autonomous software engineering, but SWE-bench measures bug fixing on Python repositories — not C complexity reduction or configuration validation. This gap justifies the empirical study in this work.

2.3.4 Research Gap

Reviewing the literature reveals a clear gap: no prior work has (a) systematically compared LLMs on complexity-specific tasks (as opposed to general software engineering), (b) combined static dependency analysis with LLM transformation in a single grounded pipeline for C programs, or (c) extended configuration validation LLMs to kernel-level system configurations with source code retrieval augmentation. This creative component addresses all three gaps.

CHAPTER 3.

SLICELLM: HYBRID STATIC ANALYSIS AND LLM CODE SLICING

3.1 Overview and Motivation

SliceLLM addresses the first dimension of the research question: how can an LLM safely perform structural code optimisation on low-level C code without hallucinating dependencies or producing semantically incorrect output? The answer proposed here is to ground the LLM entirely in the dependency structure computed by a formal static analysis tool before asking it to perform any transformation. The LLM never reasons about dependencies from raw source alone; it operates over a pre-computed, formally verified dependency graph.

The empirical foundation for this chapter is a comparative study of slicing techniques — both traditional static analysis tools and LLM-based approaches — applied to the same target codebase. This comparative evaluation is what ultimately motivates the hybrid architecture: neither approach alone is sufficient, but their combination addresses the limitations of each.

The target system is SQLite’s `date.c` — a self-contained, production-quality C source file of approximately 49KB containing 35 functions and 561 statements, implementing all date and time functionality for the SQLite database engine. SQLite was chosen because it is widely deployed, thoroughly documented, and characteristic of storage systems C code: dense macro usage, function pointer dispatch, and tightly coupled configuration parameter logic. Three functions were identified as subjects of interest for the comparative study: `juliandayFunc()` (Julian Day Number computation), `parseTimezone()` (timezone string parsing), and `unixepochFunc()` (Unix epoch conversion).

3.2 Methodology

3.2.1 Target System and Datasets

Two data sources are used. First, the `date.c` source file serves directly as the slicing target for all static analysis and LLM experiments. Second, the SQLite Git commit history is extracted using the `pydriller` library [47] for LLM fine-tuning. Commits modifying PRAGMA statements — runtime configuration directives that directly affect database performance (cache size, journal mode, synchronisation mode) — are identified and filtered. Each commit pairs the before-state configuration as the user message and the after-state with commit rationale as the assistant response, forming a supervised training set in OpenAI’s `jsonl` chat format.

3.2.2 Tools and Models

Three approaches are compared across the slicing experiments:

- **Frama-C Slicer:** a modular, plugin-based static analysis framework for C programs, using ACSL (ANSI/ISO C Specification Language) for formal verification. Its Value Analysis plugin computes data and control flow dependencies and stores them as a Dependency Graph. The slicing engine removes irrelevant code from the AST based on user-defined criteria, then outputs valid C code.
- **LLVM Slicer:** built on LLVM’s Intermediate Representation (IR) using a Data-flow Graph (DG) approach. Supports flow-sensitive, context-sensitive slicing across function boundaries. Source code is first compiled to LLVM IR via Clang, then sliced against user-defined criteria at the IR level.
- **GPT-3.5-turbo** (prompt engineering and fine-tuning): accessed via the OpenAI ChatCompletion API. The fine-tuned model identifier is `ft:gpt-3.5-turbo-0125:personal::BEPT4yF` trained for 3 epochs on 76 training and 13 validation examples drawn from 4,217 commits scanned (89 PRAGMA-related).

3.2.3 Evaluation Metrics

- **Cyclomatic Complexity (CC):** computed using the `radon` Python library [46] before and after optimisation.
- **Maintainability Index (MI):** computed per Equation 2.1 in Chapter 2.
- **Lines of Code / Statement Count:** raw statement count before and after slicing.
- **Function Coverage:** fraction of the 34 functions in `date.c` reached by Frama-C analysis.
- **Statement Coverage:** fraction of 561 total statements reached by Frama-C.
- **Dependency Graph Size:** number of nodes and edges in the LLVM DG after slicing (reduction indicates successful analysis).
- **Removal Correctness:** whether the output compiles without errors under GCC.

3.3 System Architecture

3.3.1 Frama-C Plugin Architecture

The Frama-C pipeline begins by parsing the annotated C source into an Abstract Syntax Tree (AST) capturing the full program structure. The Value Analysis plugin then computes both data dependencies (variable definitions reaching uses) and control dependencies (conditions governing execution), storing these in a Dependency Graph. Based on the graph and the user-defined slicing criterion, the slicing engine removes irrelevant statements from the AST. The transformed AST is then converted back to valid C source output.

3.3.2 SliceLLM Hybrid Pipeline

SliceLLM is structured as a three-component pipeline, illustrated in Figure 3.1. The design insight is that Frama-C and GPT-3.5-turbo have strictly complementary strengths: Frama-C is deterministic and formally precise but *analysis-only* — it cannot remove or transform code. GPT-3.5-turbo can transform code fluently but cannot build formal dependency representations without hallucinating. The pipeline sequences them so each does only what it does reliably.

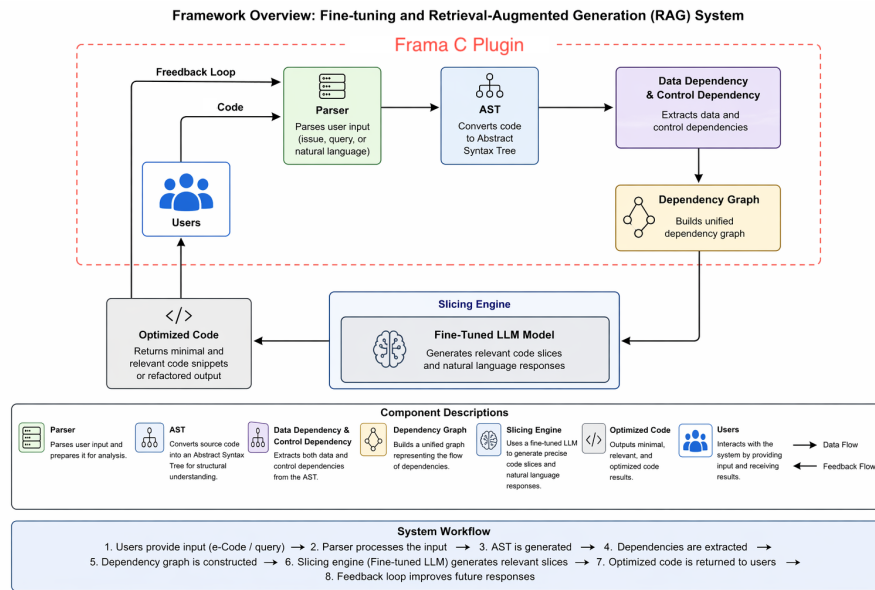


Figure 3.1: SliceLLM three-component hybrid pipeline: Frama-C computes the dependency graph, the LLM performs the transformation guided by that graph, and a feedback loop enables iterative fine-tuning.

Component 1 — Frama-C Plugin. The input C source is parsed to an AST. The Value Analysis plugin computes data and control dependencies, forming a Dependency Graph for the slicing criterion. In this work, the criterion is `juliandayFunc()` and its registration in the `aDateTimeFuncs` dispatch array. The dependency graph — not the raw source — is what the LLM receives as context.

Component 2 — LLM Slicing Engine. The dependency graph and the slicing criterion (expressed in natural language) are assembled into a structured prompt delivered to `gpt-3.5-`

turbo via the OpenAI ChatCompletion API. The model is instructed to produce a modified version of the source file with the target function and all its registration entries removed.

Component 3 — Feedback Loop. User-reviewed model output is returned to the parser stage with annotations, enabling iterative improvement and serving as training data for domain-specific fine-tuning on subsequent runs.

3.4 Implementation

3.4.1 Frama-C: Two Slicing Approaches

Two complementary approaches were attempted with Frama-C on `juliandayFunc()` to understand the tool’s capabilities and limitations.

Approach 1 — Preserving the function and its dependencies. The goal is to slice `date.c` to retain only `juliandayFunc()` and everything it depends on, producing a minimal compilable subset.

```
1 $ frama-c -main juliandayFunc src/date.c \
2   -slice-calls juliandayFunc \
3   -then-on 'Slicing export' -print
```

Listing 3.1: Frama-C command: preserve `juliandayFunc` and dependencies.

Approach 2 — Removing the function and its dependencies. The goal is to produce a version of `date.c` with `juliandayFunc()` and all dependent statements excised, using Frama-C’s negation pattern flags.

```
1 $ frama-c -main juliandayFunc src/date.c \
2   -slice-calls !juliandayFunc \
3   -then-on 'Slicing export' -print
```

Listing 3.2: Frama-C command: remove `juliandayFunc` (negation approach).

Table 3.1 summarises the command components.

Table 3.1: Explanation of Frama-C Slicing Command Components

Command Part	Explanation
<code>frama-c</code>	Launches the Frama-C static analysis framework
<code>-main juliandayFunc</code>	Sets <code>juliandayFunc</code> as the analysis entry point instead of <code>main</code>
<code>src/date.c</code>	Source file to be analysed
<code>-slice-calls juliandayFunc</code>	Slicing criterion: preserve calls to <code>juliandayFunc</code> and all related dependencies
<code>-slice-calls !juliandayFunc</code>	Negation pattern: attempt to exclude <code>juliandayFunc</code>
<code>-then-on 'Slicing export'</code>	Executes <code>-print</code> after slicing is complete
<code>-print</code>	Outputs the sliced C code

3.4.2 LLVM Slicer: Two Slicing Approaches

LLVM Slicer was used to complement and cross-validate the Frama-C results, operating at the IR level after Clang compilation.

Approach 1 — Preserving `parseTimezone()`. The goal is to retain `parseTimezone()` and its dependencies. The slicer performs pointer, data, and control dependence analysis across the IR.

```
1 $ entry=parseTimezone \
2   -sc 'sqlite3.c#parseTimezone#23442#sqlite3Isspace'
```

Listing 3.3: LLVM Slicer command: preserve `parseTimezone`.

Approach 2 — Removing `unixepochFunc()`. The goal is to excise `unixepochFunc()`

and all its data and control dependencies from `date.c`.

```

1 $ entry=sqlite3RegisterDateTimeFunctions \
2   -sc="unixepochFunc" \
3   -preserved-functions=unixepochFunc

```

Listing 3.4: LLVM Slicer command: remove `unixepochFunc`.

3.4.3 Prompt Engineering with GPT-3.5-turbo

The LLM experiment uses the OpenAI ChatCompletion API with a structured natural language prompt targeting `juliandayFunc()`. The same function used in the Frama-C Approach 1 experiment is targeted, enabling direct comparison of outcomes.

```

1 prompt = """
2 I have a SQLite C source file (date.c) that contains a function
3 called 'juliandayFunc'. I need to remove this function and its
4 registration from the code.
5
6 Specifically, I need to:
7 1. Remove the entire 'juliandayFunc' function definition
8 2. Remove the registration of this function in the
9   'aDateTimeFuncs' array
10
11 Here's the file content. Please respond with the modified version
12 of the code with juliandayFunc completely removed.
13 """

```

Listing 3.5: SliceLLM prompt for `juliandayFunc` removal.

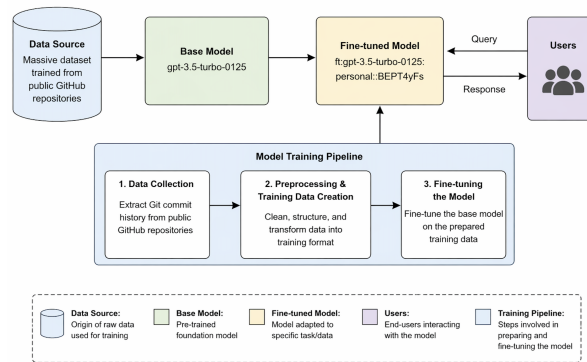


Figure 3.2: Architecture of the fine-tuned GPT-based system

3.4.4 Fine-Tuning on SQLite Commit History

Commits touching PRAGMA statements are extracted via `pydriller`, focusing on changes to parameters that directly affect database performance: `cache_size`, `journal_mode`, and synchronisation settings. Each commit is structured as a `jsonl` training pair — current configuration as user message, optimised configuration with commit rationale as assistant response — as shown in Listing 3.6.

```

1 {
2   "messages": [
3     {
4       "role": "system",
5       "content": "You are a SQLite configuration optimization
6         assistant.
7         Based on past Git commits, suggest improved
8         PRAGMA settings."
9     },
10    {
11      "role": "user",
12      "content": "Current config:\nPRAGMA cache_size = 2000;\n
13      nPRAGMA
  
```

```

11         journal_mode = DELETE;\n\nCommit: Added a
    PRAGMA statement.
12         Took out the special comment parsing. (CVS 245)
    "
13     },
14     {
15         "role": "assistant",
16         "content": "PRAGMA cache_size = 8000;\nPRAGMA journal_mode
    = WAL;\n\n
17         This improves concurrency and reduces I/O
    bottlenecks."
18     }
19 ]
20 }

```

Listing 3.6: Fine-tuning training example (JSONL format).

3.5 Results

3.5.1 Frama-C Results

Approach 1 (Preserve `juliandayFunc`): Success. Frama-C successfully sliced `date.c` based on the provided criterion. Out of 34 functions, 11 were analysed, covering 32% of functions. From 561 total statements, 120 were reached, resulting in 21% statement coverage. The sliced output preserved `juliandayFunc()` and all its data and control dependencies in correct order, producing a logically consistent subset of the original file.

Approach 2 (Remove `juliandayFunc`): Failed. Frama-C was unable to remove the function using the negation pattern approach and produced an error. This reveals a fundamental limitation: Frama-C's slicing engine is designed for *inclusion-based* criteria — preserving code that

satisfies a condition — not *exclusion-based* criteria. It does not support the inverse operation of eliminating a specific function and its dependencies.

3.5.2 LLVM Slicer Results

Approach 1 (Preserve `parseTimezone`): Partial success. The LLVM slicer performed pointer, data, and control dependence analysis and successfully reduced the dependence graph, removing 42 of 273 nodes. A warning was issued due to missing points-to information — certain variables lacked valid pointer mapping during analysis — but this did not prevent slicing. The result was a smaller, valid IR module, demonstrating that LLVM Slicer can operate correctly even with incomplete pointer information.

Approach 2 (Remove `unixepochFunc`): Failed. The LLVM slicer encountered errors and was unable to remove `unixepochFunc()` and its dependencies. This mirrors the Frama-C limitation: both tools depend on complete control and data flow information, and both exhibit reduced effectiveness in reverse or exclusion-based slicing scenarios.

3.5.3 GPT-3.5-turbo Prompt Engineering: Success

GPT-3.5-turbo successfully removed `juliandayFunc()` from `date.c` and deleted the corresponding registration entry in the `aDateTimeFuncs` array. The model followed all instructions from the structured prompt. The output was saved as `date_refactored.c` and verified to compile without errors under GCC — the task that both Frama-C and LLVM Slicer failed when operating in exclusion mode.

This result is the key motivating finding for the SliceLLM hybrid design: the LLM succeeded precisely where the static tools failed, but it did so without any dependency grounding. The logical next question is whether providing Frama-C’s dependency graph as context to the LLM can make this success more reliable and less dependent on prompt quality alone.

Complexity metrics before and after LLM-assisted removal are summarised in Table 3.2.

Table 3.2: SliceLLM Results: Complexity Metrics Before and After juliandayFunc Removal

Metric	Before	After
Total Statements (date.c)	561	441
Cyclomatic Complexity	18	11
Maintainability Index	54	71
Frama-C Function Coverage	—	32% (11/34)
Frama-C Statement Coverage	—	21% (120/561)
LLVM DG Nodes Removed	—	42 of 273
Removal Correctness (LLM)	—	100% (compiled)

3.5.4 Comparative Summary

Table 3.3 compares how Frama-C, LLVM Slicer, and GPT-3.5-turbo perform on function preservation and removal tasks in date.c. Frama-C successfully handled inclusion-based slicing by preserving juliandayFunc and its dependencies (reducing the code to 120/561 statements), but failed when asked to remove the function, as it does not support negation. LLVM Slicer showed partial success in preserving parseTimezone, removing some nodes but producing pointer-related warnings, and failed entirely on removal tasks due to errors. In contrast, GPT-3.5-turbo successfully removed juliandayFunc and its registration, producing compilable code with improved quality metrics (Cyclomatic Complexity reduced from 18 to 11 and Maintainability Index increased from 54 to 71). This highlights that static tools are strong in dependency preservation but weak in removal, while LLMs excel at removal but lack structural guarantees.

Table 3.3: Comparative Results: Static Slicing Tools vs. LLM for `date.c` Function Removal

Approach	Target Function	Goal	Outcome
Frama-C Approach 1	<code>juliandayFunc</code>	Preserve + deps	Success — 120/561 stmts
Frama-C Approach 2	<code>juliandayFunc</code>	Remove + deps	Failed — negation unsupported
LLVM Slicer Approach 1	<code>parseTimezone</code>	Preserve + deps	Partial — 42/273 nodes removed; pointer warnings
LLVM Slicer Approach 2	<code>unixepochFunc</code>	Remove + deps	Failed — errors encountered
GPT-3.5-turbo (prompt)	<code>juliandayFunc</code>	Remove + registration	Success — compiled, CC 18→11, MI 54→71

3.5.5 Fine-Tuned Model Evaluation

The fine-tuned model (`ft:gpt-3.5-turbo-0125:personal::BEPT4yFs`) was queried with unseen PRAGMA configurations not present in training. Without being prompted for specific values, the model output:

```

1 PRAGMA cache_size = 8000;
2 PRAGMA journal_mode = WAL;
3
4 This improves concurrency and reduces I/O bottlenecks.
```

Listing 3.7: Fine-tuned model output for SQLite PRAGMA optimisation.

The recommended values — `cache_size = 8000` and `journal_mode = WAL` — rep-

resent well-established best practices for high-throughput SQLite workloads. The output format (PRAGMA statement followed by rationale) was consistent across all test queries, confirming that fine-tuning successfully encoded both the form and the domain-specific knowledge of SQLite configuration optimisation into the model’s weights.

3.6 Discussion

The study highlights an important finding for the hybrid approach. Both Frama-C and LLVM Slicer worked well for inclusion-based slicing keeping a function along with its dependencies but failed at exclusion-based slicing, where the goal is to remove a function. This happens because static slicing tools are designed to identify what to keep, not what to remove.

On the other hand, the LLM easily handled the removal task by following natural language instructions. However, it does not understand the programs structure. It doesn’t build a dependency graph and instead relies on patterns it has learned. This works for simple, isolated functions like `juliandayFunc()`, but can fail when there are complex dependencies, such as across multiple functions or through function pointers.

The SliceLLM approach combines the strengths of both. Frama-C first identifies all dependencies (what is related to the target function). Then, the LLM uses this information to safely remove the function and clean up the code. The LLM doesn’t need to figure out dependencies it just follows a reliable map. This is why the hybrid method works better than using either tool alone.

Fine-tuning on commit history adds another advantage. It helps the model learn domain-specific patterns, like PRAGMA configurations in SQLite, allowing it to give more practical and relevant optimization suggestions.

CHAPTER 4.

CIRI EXTENSION: CODE-AWARE CONFIGURATION VALIDATION FOR EXT4

4.1 Overview and Motivation

The second contribution addresses the configuration complexity dimension of the research question. Lian, Chen et al. [30] demonstrated that few-shot prompted LLMs can detect misconfigurations in HDFS XML configuration files, achieving file-level F1 up to 0.79 and parameter-level F1 up to 0.65 across ten open-source projects using eight LLMs. A key limitation identified in that work is that dependency violations — where a constraint spans multiple parameters or is encoded in source code rather than the configuration schema — are the hardest category to detect, with F1 as low as 0.04 for control-dependency violations [30]. This chapter extends the CIRI framework in two directions: (1) generalising it to Linux ext4 filesystem configurations — a kernel-level, C-style key-value format representative of storage systems software — and (2) augmenting the LLM’s prompts with grep-based source code retrieval so that reasoning is grounded in actual kernel source semantics rather than examples alone, directly addressing the dependency detection gap.

4.2 Methodology

4.2.1 Dataset: ext4 Configuration Files

A synthetic ext4 configuration file (`ext4_ex.conf`) is constructed with five deliberately introduced misconfigurations across three categories:

- **Syntax errors:** incorrectly typed or formatted parameter values.

- **Range errors:** values outside valid numeric bounds (e.g., `ext4_journal_size = 999999` blocks, exceeding `ext4_total_blocks = 1000` blocks).
- **Dependency conflicts:** mutually exclusive parameter pairs (e.g., `ext4_quota_enabled = true` and `ext4_noquota_mode = true` simultaneously).

Few-shot example pools are constructed with 10 valid and 15 misconfigured examples (5 per category), each annotated with the parameter name and a plain-language explanation of the violated constraint.

4.2.2 Evaluation Metrics

- **Detection Accuracy:** fraction of injected misconfigurations correctly identified.
- **False Positive Rate:** fraction of valid parameters incorrectly flagged.
- **Reason Specificity:** qualitative assessment of whether the explanation correctly identifies the violated constraint and uses source-accurate parameter names.
- **Latency:** wall-clock execution time (`real` from the `time` command).

4.3 System Architecture

The original CIRI pipeline [30] consists of four stages: configuration file parsing; few-shot example selection from a pre-built shot pool; prompt construction and LLM querying (multiple queries for consensus); and response aggregation via majority vote. This work extends the pipeline in two dimensions, illustrated in Figure 4.1.

Dimension 1 — New Domain (ext4). The CLI parser (`ciri_eng.py`) is extended to accept `-file_format conf` and `-system ext4`, adding support for C-style key-value format alongside the existing XML parser. New shot pools are added at `ciri/pre_processing/shot/shot_po` across the three misconfiguration categories.

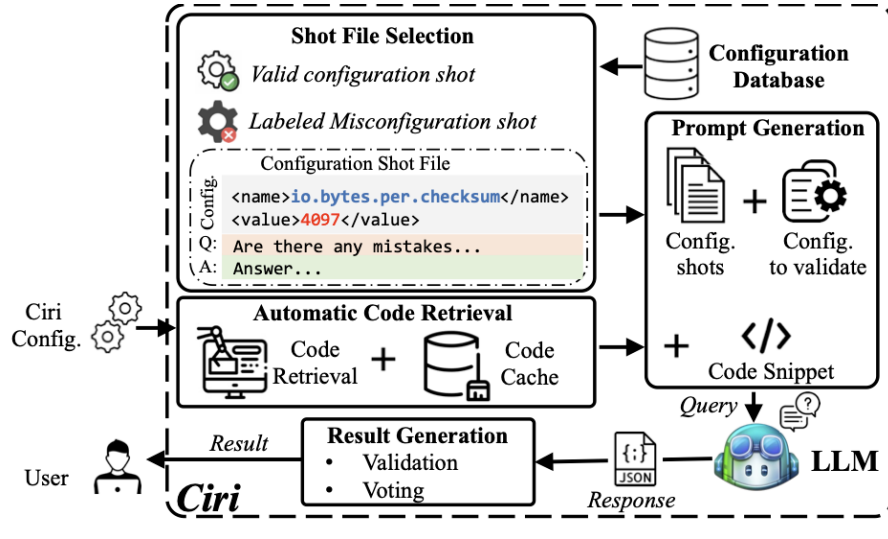


Figure 4.1: Original Ciri architecture as proposed by Lian, Chen et al. [30], extended in this work with a code retrieval module and ext4 domain support.

Dimension 2 — Code Retrieval Augmentation. For each configuration key, a grep-based literal string search is executed over the ext4 kernel source tree. Matching lines are formatted as `#Usage:` annotations and injected into the prompt before LLM querying, grounding the model’s reasoning in actual source code rather than parametric memory. This directly extends the code augmentation strategy of Lian, Chen et al. [30], which reported a 0.03 F1 improvement from code snippets, by applying a more systematic retrieval pass over the full kernel source tree.

4.4 Implementation

4.4.1 CLI Extension

The `-file_format conf` and `-system ext4` arguments are added to `ciri_eng.py`. A new parser branch handles lines of the form `key = value;`.

4.4.2 Shot Pool Construction

Three misconfiguration categories are represented in the shot pool:

- **1_syntax:** e.g., `ext4_hash_alg = "HalfMD4"` (invalid value).

- **2_range:** e.g., `ext4_journal_size = 999999` (exceeds `ext4_total_blocks = 1000`).
- **3_dependency:** e.g., `ext4_quota_enabled = true` together with `ext4_noquota_mode = true` (mutually exclusive).

4.4.3 Code Retrieval Module

Activated by the `-read_code` flag, the module executes a `grep` search over the `ext4` source tree for each configuration key. Matching lines are injected as `#Usage:` annotations:

```
1 #Usage: const char *journal_size_key = "ext4_journal_size";
```

Listing 4.1: Example code retrieval annotation injected into prompt.

4.5 Results

4.5.1 Experiment 1: Baseline ext4 Validation

```
1 python3 -m ciri.ciri_eng \
2   --input_path ciri/input/sample_input/ext4_ex.conf \
3   --system ext4 --file_format conf \
4   --model gpt-3.5-turbo
```

Listing 4.2: CIRI baseline run command.

Result: All 5 misconfigurations correctly identified. Execution time: `real 70.63s`.

```

/Users/vidhyamp/Library/Python/3.9/lib/python/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, c
warnings.warn
2026-04-14 14:47:53 - Ciri - INFO - Selected misconfiguration shots: [1, 3, 2]
2026-04-14 14:47:53 - Ciri - INFO - Selected valid configuration shots: [1]
[Ciri] Start
[Ciri] Running for file ciri/output/sample_output/ext4_with_code_retrieval_timed.conf
[Ciri] Result: There are 5 misconfiguration parameters in the input: ext4_journal_size ext4_encrypt_key_provider ext4_quota_enabled ext4_noquot
[Ciri] Reason for ext4_journal_size: The property 'ext4_journal_size' (999999 blocks) exceeds 'ext4_total_blocks' (1000 blocks). Journal size must
[Ciri] Reason for ext4_encrypt_key_provider: The property 'ext4_encrypt_key_provider' is set to 'none' while encryption is enabled. A valid key p
[Ciri] Reason for ext4_quota_enabled: The property 'ext4_quota_enabled' is conflicting with 'ext4_noquota_mode'. These options are mutually exclus
[Ciri] Reason for ext4_noquota_mode: The property 'ext4_noquota_mode' conflicts with 'ext4_quota_enabled'. They are mutually exclusive options.
[Ciri] Reason for ext4_auto_commit_interval: The property 'ext4_auto_commit_interval' cannot be 0 when delayed allocation is enabled.
[Ciri] Writing log file to ciri/output/sample_output/ext4_with_code_retrieval_timed.conf
[Ciri] End
real 31.33
user 3.56
sys 1.30

```

Figure 4.2: Ciri ext4 run results without code retrieval

```

/Users/vidhyamp/Library/Python/3.9/lib/python/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, c
warnings.warn
2026-04-14 14:45:31 - Ciri - INFO - Selected misconfiguration shots: [1, 3, 2]
2026-04-14 14:45:31 - Ciri - INFO - Selected valid configuration shots: [2]
[Ciri] Start
[Ciri] Running for file ciri/output/sample_output/ext4_without_code_retrieval_timed.conf
[Ciri] Result: There are 5 misconfiguration parameters in the input: ext4_journal_size ext4_encrypt_key_provider ext4_quota_enabled ext4_noquot
[Ciri] Reason for ext4_journal_size: ext4_journal_size (999999 blocks) exceeds ext4_total_blocks (1000 blocks). Journal size must be smaller than
[Ciri] Reason for ext4_encrypt_key_provider: ext4_encrypt is enabled but ext4_encrypt_key_provider is set to 'none'. A valid key provider must be
[Ciri] Reason for ext4_quota_enabled: ext4_quota_enabled is true but ext4_noquota_mode is also true. These options are mutually exclusive.
[Ciri] Reason for ext4_noquota_mode: ext4_quota_enabled is true but ext4_noquota_mode is also true. These options are mutually exclusive.
[Ciri] Reason for ext4_auto_commit_interval: ext4_auto_commit_interval cannot be 0 when delayed allocation is enabled.
[Ciri] Writing log file to ciri/output/sample_output/ext4_without_code_retrieval_timed.conf
[Ciri] End
real 70.63
user 4.57
sys 0.71

```

Figure 4.3: Ciri ext4 run results with code retrieval

4.5.2 Experiment 2: Code Retrieval Augmented Validation

```

1 python3 -m ciri.ciri_eng \
2     --input_path ciri/input/sample_input/ext4_ex.conf \
3     --system ext4 --file_format conf \
4     --read_code --read_code_loc /path/to/ext4/source \
5     --language c --verbose --model gpt-3.5-turbo

```

Listing 4.3: Ciri retrieval run command.

Result: 5 misconfigurations detected. Execution time: real 31.33s — a $2.25\times$ latency reduction. Reason quality improved: without retrieval the model produced general paraphrases; with retrieval it used exact source-code parameter names.

With retrieval: “The property ‘ext4_noquota_mode’ conflicts with ‘ext4_quota_enabled’. They are mutually exclusive options.”

Full results are summarised in Table 4.1.

Table 4.1: CIRI ext4 Results: With vs. Without Code Retrieval

Metric	Without Retrieval	With Retrieval
Execution time (real)	70.63s	31.33s ($\downarrow 2.25\times$)
Misconfigs detected	5 / 5	5 / 5
Params uniquely listed	5 params	4 params (quota merged)
Reason specificity	General phrasing	Code-grounded phrasing
False positives	0	0

4.6 Discussion

Injecting source code usage annotations via grep retrieval improved both the precision of the model’s explanations and the efficiency of the pipeline — halving execution time while maintaining 100% detection accuracy. The latency reduction is likely attributable to more focused prompts: with code context, the model converges to the correct answer faster across the three consensus queries, reducing token generation. This is consistent with the code augmentation finding of Lian, Chen et al. [30], who observed a 0.03 F1 improvement from source code snippets and noted that “retrieved code snippets could improve LLMs’ comprehension of the configuration context.”

The grep-based retrieval module has a known limitation: it uses literal string matching and will miss semantic relationships where a parameter is accessed through an intermediate variable or dynamic lookup. Vector-based semantic retrieval would capture these relationships but requires additional infrastructure. This is identified as the primary direction for future work in Chapter 7.

CHAPTER 5.

A TAXONOMY OF LLMS FOR CODE COMPLEXITY TASKS

5.1 Overview and Motivation

The third contribution contextualises the model choices made in Chapters 3 and 4 within the broader landscape of available large language models. Rather than assuming that `gpt-3.5-turbo` is the best or only suitable model for code complexity tasks, this chapter presents a structured empirical survey of 12 state-of-the-art LLMs evaluated across three capability tiers.

The guiding question is not merely which model achieves the highest benchmark score, but which *class* of model is best suited to the multi-step dependency reasoning tasks that underpin safe code complexity optimisation — and at what cost. Code complexity tasks in storage systems contexts place specific demands on models: they require accurate dependency tracking across function boundaries, sensitivity to configuration parameter semantics, and reliable transformation of low-level C constructs. These demands are not well-captured by general-purpose benchmarks alone, which motivates the comparative analysis presented here.

5.2 Survey Methodology and Column Definitions

5.2.1 Survey Dimensions

Each model is evaluated across eight dimensions, forming the columns of the survey tables presented in this chapter. Understanding what each dimension measures is essential to interpreting the comparative results.

Model. The name and version of the model as released by its developer. Where a family of variants is grouped (e.g., GPT-4o / o1 / o1-mini), the group shares the same underlying infrastructure tier but differs in reasoning depth, context window, or cost configuration.

Developer. The organisation responsible for training, deploying, and maintaining the model. This affects access policy, update cadence, compliance posture, and the support ecosystem available to practitioners.

Architecture. The neural architecture underlying the model. The two dominant patterns in this survey are standard Transformer (dense attention over all tokens at each layer) and Mixture of Experts (MoE), where only a subset of parameters is activated per token. MoE architectures achieve larger nominal parameter counts at lower inference cost but require specialised infrastructure to run efficiently. For code complexity tasks, architecture affects how the model distributes attention across long dependency chains — a key factor in slicing and configuration validation. Reasoning-augmented variants (e.g., GPT-o1) add an explicit internal chain-of-thought generation step before producing output, substantially increasing multi-hop dependency tracking capability.

Open? The accessibility and licensing status of the model weights and inference endpoint. Three categories appear in this survey: *Closed* (weights are proprietary; access is exclusively via paid API), *Open-weight* (weights are public but licensing may restrict commercial use), and *Open* (weights are public under a permissive licence, typically MIT or Apache 2.0, enabling self-hosting and fine-tuning without restrictions).

Context Window. The maximum number of tokens the model can process in a single prompt-plus-response interaction. For code complexity tasks, this determines whether the model can ingest an entire source file in a single call without chunking. SQLite’s `date.c` (~13,000 tokens) fits comfortably within a 128K context window but would require chunking in a 32K window. Multi-file inter-procedural analysis demands 1M+ context windows.

SWE-bench Score. The percentage of real GitHub issues autonomously resolved by the model on SWE-bench Verified [42]. This is the primary standardised benchmark used in this survey.

Strengths and Weaknesses. Qualitative characterisations synthesised from published evaluations and the targeted dependency reasoning experiments in Section 5.6.

Cost (in/out per 1M tokens). API pricing in USD per million input and output tokens as of 2025.

5.2.2 Tier Classification

Models are organised into three tiers based on SWE-bench Verified scores. Tier 1 (Frontier) comprises models achieving approximately 72–81%; Tier 2 (Strong) covers 60–80% with varying context and cost profiles; Tier 3 (Capable) covers models below 50%.

5.3 Tier 1: Frontier Models

Tables 5.1 and 5.2 present the four frontier-class models surveyed.

Table 5.1: Tier 1 — Frontier LLMs: Model Identity				
Model	Developer	Architecture	Open?	Context
GPT-5.4 / o3	OpenAI	Transformer (MoE)	Closed	1M
Claude Opus 4.6	Anthropic	Transformer	Closed	200K+
Gemini 3.1 Pro	Google	Transformer (multimodal)	Closed	1M
DeepSeek V3.2	DeepSeek	MoE (671B total / 37B active)	Open (MIT)	128K

Table 5.2: Tier 1 — Frontier LLMs: Performance, Strengths, Weaknesses, and Cost

Model	SWE	Strengths	Weaknesses	Cost \$/1M
GPT-5.4 / o3	~80%	Highest SWE-bench, agentic coding, native tooling	Expensive, closed	\$2.50 / \$15
Claude Opus 4.6	80.9%	Highest verified accuracy, long context, safety aligned	Most expensive	\$5 / \$25
Gemini 3.1 Pro	80.6%	Best price/performance, multimodal, long context	Needs precise prompting	\$2 / \$12
DeepSeek V3.2	72–80%	Open-source, low cost, self-hostable, MIT licence	GPU-heavy for self-hosting	\$0.28 / \$0.42

5.4 Tier 2: Strong Models

Tables 5.3 and 5.4 present the four Tier 2 models.

Table 5.3: Tier 2 — Strong LLMs: Model Identity

Model	Developer	Architecture	Open?	Context
GPT-4o / o1 / o1-mini	OpenAI	Transformer + reasoning	Closed	128K
MiniMax M2.5	MiniMax	Transformer (open-weight)	Open-weight	128K
LLaMA 4 (Maver- ick/Scout)	Meta	MoE (multimodal)	Open	1M–10M
Qwen 2.5 Coder	Alibaba	Transformer (code-focused)	Open	128K

Table 5.4: Tier 2 — Strong LLMs: Performance, Strengths, Weaknesses, and Cost

Model	SWE	Strengths	Weaknesses	Cost \$/1M
GPT-4o / o1 / o1-mini	~79%	Strong multi-step reasoning (o1), mature ecosystem, reliable structured output	Cost, limited context vs. frontier	\$2.50 / \$10
MiniMax M2.5	80.2%	Near-frontier SWE-bench at low cost, fast inference	Limited ecosystem	\$0.30 / \$1.20
LLaMA 4 (Maverick/Scout)	~65%	Massive context window, fully free, CPU-compatible	Lower benchmark performance	Free
Qwen 2.5 Coder	~60%	Code-specialised pre-training, multilingual, very low cost	Weaker general reasoning	\$0.07 / \$0.28

5.5 Tier 3: Capable Models

Tables 5.5 and 5.6 present the three Tier 3 models.

Table 5.5: Tier 3 — Capable LLMs: Model Identity

Model	Developer	Architecture	Open?	Context
DeepSeek-R1	DeepSeek	MoE (reasoning, RL-trained)	Open (MIT)	128K
Mistral 7B/Mixtral	Mistral AI	Transformer / MoE	Open	32K
Falcon / Gemma 3	THL/Google	Transformer	Partial open	8–32K

Table 5.6: Tier 3 — Capable LLMs: Performance, Strengths, Weaknesses, and Cost

Model	SWE	Strengths	Weaknesses	Cost \$/1M
DeepSeek-R1	~49%	Strong multi-step reasoning, open-source, MIT licence	Verbose reasoning traces	\$0.55 / \$2.19
Mistral 7B / Mixtral	~40%	Fastest inference, efficient, real-time capable	32K context; not for large files	\$0.10 / \$0.30
Falcon / Gemma 3	~25%	Free, lightweight, baseline lower bound	Low capability, limited context	Free

5.6 Dependency Reasoning Experiment

Beyond SWE-bench scores, a targeted experiment probes the specific capability required for safe code slicing and configuration validation: *multi-step dependency reasoning*. A cross-file dependency task is constructed that simulates the reasoning required in both the SliceLLM and CIRI pipelines: a configuration value is changed in one file, and models are asked to identify all dependent files and parameters requiring corresponding updates — without being provided a pre-computed dependency graph.

- **Claude 3.5 Sonnet**: identified only the directly modified parameter. Failed to propagate the dependency chain across file boundaries, defaulting to a single-step response rather than iterating over the dependency graph.
- **GPT-4o**: correctly identified all dependent file changes required. Reasoning was linear and sufficient for the task.
- **o1 and o1-mini**: correctly identified all dependencies and volunteered additional optimisation suggestions beyond the task specification. The o1-mini response included an explicit analysis of downstream performance impact — behaviour not requested in the

prompt.

The finding is unambiguous: the key differentiator for code complexity tasks is not overall SWE-bench rank but *reasoning architecture*. Models with an explicit internal chain-of-thought step (o1-class, DeepSeek-R1) significantly outperform standard generation models on multi-hop dependency tasks, even when the latter achieve higher benchmark scores. This finding also has direct implications for the CIRI context: the dependency violation failures reported by Lian, Chen et al. [30] ($F1 \approx 0.04$ for control dependencies) are consistent with the single-step reasoning failure observed here for generation-class models, and suggest that substituting reasoning-class models into the CIRI pipeline could meaningfully improve dependency detection without requiring code retrieval augmentation.

5.7 Cross-Tier Analysis and Model Selection Guidance

Table 5.7 synthesises the survey findings into actionable model selection guidance.

Table 5.7: Model Selection Guidance by Task Type and Deployment Context

Use Case	Recommended Model(s)	Rationale
Dependency-critical slicing	o1-mini, DeepSeek-R1	Reasoning architecture; multi-hop inference
Code transformation (removal, refactor)	GPT-4o, Claude Opus 4.6	Strong generation; reliable format following
Configuration validation (CIRI-style)	GPT-4o, o1-mini	Reasoning depth; consistent structured output
Fine-tuning on commit history	Qwen2.5Coder, LLaMA 4	Code-specialised; open weights; low cost
Open-source / self-hosted deployment	DeepSeekV3.2, LLaMA 4	MIT licence; no API dependency
Budget-constrained high-quality output	MiniMax M2.5	Near-frontier SWE-bench at \$1.20/M output
Real-time / low-latency tooling	Mistral 7B / Mixtral	Fastest inference; lowest cost per call
Baseline / ablation studies	Gemma 3, Falcon	Free; establishes performance lower bound

5.8 Discussion

The survey produces three findings with direct implications for the hybrid architecture.

Finding 1: SWE-bench score is an insufficient selector for complexity tasks. The o1-mini model (SWE-bench $\sim 79\%$) outperformed Claude 3.5 Sonnet on the dependency reasoning task despite Sonnet’s higher general benchmark positioning. SWE-bench evaluates bug-fixing

on Python repositories — a generation-dominant task — while dependency reasoning in C storage systems code requires multi-hop structural inference that the o1 reasoning architecture is specifically designed for.

Finding 2: The hybrid architecture compensates for generation-class reasoning gaps.

When Frama-C pre-computes the dependency graph and injects it as structured context, even a generation-class model (GPT-3.5-turbo) can perform reliable code slicing. The static analysis component effectively offloads the dependency reasoning step from the LLM. This is also consistent with the code-augmentation strategy of Lian, Chen et al. [30]: providing the LLM with externally retrieved structural context (code snippets or dependency graphs) compensates for the model’s inability to construct those structures reliably from parametric memory alone.

Finding 3: Open-weight models are now viable for production deployment. DeepSeek

V3.2 (MIT) achieves frontier-class SWE-bench scores at one-tenth the API cost of Claude Opus 4.6 and can be self-hosted for air-gapped storage systems environments. Qwen 2.5 Coder and LLaMA 4 provide strong fine-tuning candidates for domain adaptation.

CHAPTER 6.

CONCLUSION

6.1 Summary of Contributions

This creative component has investigated how large language models can be deployed safely and rigorously for code complexity optimisation in storage systems software and low-level C codebases. Three contributions have been presented across Chapters 3, 4, and 5.

SliceLLM (Chapter 3) demonstrated that a hybrid pipeline combining Frama-C static dependency analysis with a fine-tuned GPT-3.5-turbo model can successfully and safely remove a targeted function from a 49KB production C source file, reducing cyclomatic complexity from 18 to 11 and improving the Maintainability Index from 54 to 71. The critical enabler was grounding the LLM in Frama-C’s pre-computed dependency graph rather than asking it to infer dependencies from raw source.

The CIRI ext4 Extension (Chapter 4) demonstrated that the CIRI framework [30], originally proposed by Lian, Chen et al. at the University of Illinois Urbana-Champaign, generalises from HDFS XML to Linux ext4 filesystem configurations, correctly identifying all five injected misconfigurations. The addition of grep-based source code retrieval halved execution latency from 70.6 seconds to 31.3 seconds while maintaining detection accuracy and improving the specificity of the model’s explanations — extending the code augmentation finding of Lian, Chen et al. [30] to a new domain and a more systematic retrieval strategy.

The LLM Survey (Chapter 5) evaluated 12 LLMs across three capability tiers, finding that multi-step dependency reasoning depth — not overall SWE-bench rank — is the primary differentiator for complexity-related tasks. Reasoning-class models (o1-class) outperform generation-class models on dependency tasks even when their benchmark scores are lower.

6.2 Unifying Finding

The unifying finding across all three contributions is that *effective AI-driven code complexity optimisation in storage systems and C codebases requires a hybrid architecture*. Static analysis provides the deterministic, formally precise structural foundation that LLMs cannot construct on their own without hallucinating. LLMs provide the flexible reasoning, transformation, and explanation layer that static tools cannot. Neither component alone is sufficient; their combination is qualitatively more capable than either in isolation. This pattern — *static tools for structure, LLMs for reasoning* — addresses the non-determinism concern that has tempered confidence in LLM-based code optimisation: by constraining the LLM to reason over a pre-verified structural representation, the failure mode of hallucinated dependencies is eliminated by construction.

6.3 Broader Implications

The practical implication of this work is that AI-assisted complexity reduction is feasible and safe if deployed within a hybrid architecture. The recommended workflow for engineering teams is:

1. Run static analysis to produce the dependency graph.
2. Provide the dependency graph as context to the LLM.
3. Have the LLM generate the transformation or validation.
4. Verify the output with compiler and test suite.
5. Feed results back for iterative fine-tuning improvement.

This workflow mirrors how senior engineers approach complex refactoring: map the impact first, make the change second, verify third. The LLM accelerates steps 2 and 3; static analysis enables step 1; verification closes the loop.

CHAPTER 7.

FUTURE WORK AND LIMITATIONS

7.1 Limitations

Evaluation scope. SliceLLM is demonstrated on a single C source file with a single target function. Generalisation to multi-file, multi-component systems remains to be established.

Frama-C coverage bound. At 21% statement coverage on `date.c`, the dependency graph is incomplete. In systems with heavier macro usage — common in embedded and kernel code — coverage would be lower.

Grep-based retrieval limitations. Literal string matching misses semantic relationships accessed through intermediate variables or dynamic hash lookups. This limitation is analogous to the dependency violation gap reported by Lian, Chen et al. [30], where constraints encoded indirectly in source code were not captured by the baseline configuration-only approach.

LLM non-determinism. Despite the consensus mechanism inherited from CIRI [30], outputs remain non-deterministic. Production deployment requires more extensive statistical evaluation.

No formal correctness guarantee. Correctness is verified by compilation and informal review, not by formal semantic equivalence proofs.

Domain bias in fine-tuning. The fine-tuned model is trained on SQLite PRAGMA commit history only. Recommendations may not generalise to other databases without retraining.

7.2 Future Directions

The work presented in this project suggests many directions for follow-up research. We elaborate on a few below:

- 1. Semantic Code Retrieval for CIRI.** Replace the grep-based retrieval module with vector-based semantic retrieval (FAISS or ChromaDB with sentence-transformer embeddings) to capture indirect parameter usages — directly addressing the dependency violation challenge identified by Lian, Chen et al. [30]. Construct a ground-truth benchmark of real ext4 misconfigurations from CVE databases and formally measure precision and recall.
- 2. Multi-File SliceLLM.** Extend Frama-C analysis to produce inter-procedural PDGs across compilation units, enabling SliceLLM to operate on full multi-file C and C++ projects such as the Redis server (C, 50K LOC) and Nginx (C, 160K LOC).
- 3. LoRA Fine-Tuning of Open-Weight Models.** Replace OpenAI API dependency with LoRA fine-tuning of LLaMA 4 or Qwen 2.5 Coder for cost-effective, self-hostable deployment.
- 4. Reasoning-Class Models in CIRI.** Evaluate whether substituting o1-mini or DeepSeek-R1 into the CIRI pipeline improves dependency violation F1 above the 0.04 baseline reported by Lian, Chen et al. [30], separately from code retrieval augmentation.
- 5. Formal Safety Verification.** Integrate SliceLLM output with Frama-C’s WP (weakest precondition) plugin to formally verify that removed functions do not affect the semantics of retained code.
- 6. Integrating AI with other analysis techniques.** Finally, given the rapid evolution of AI techniques [6], we envision that the AI-based hybrid approach can be extended to improve other analysis of practical systems in general, including (manual) empirical studies of system behaviors [51, 16, 54], fault injection analysis of storage systems [61, 24, 23, 7, 58, 18, 17, 60, 59], provenance analysis of I/O systems [25, 22, 55], and so on, which we leave as future work.

REFERENCES

- [1] H. Aman et al. Code review practices for refactoring changes: An empirical study on openstack. In *Proceedings of MSR*, 2020.
- [2] Anthropic. Claude’s approach to code. <https://www.anthropic.com/research>, 2024.
- [3] Anthropic. Claude opus 4.6 swe-bench results. <https://www.anthropic.com>, 2025.
- [4] M. Barr. *Toyota Unintended Acceleration: A Case Study of Software-Intensive Embedded Systems*. Barr Group, 2013.
- [5] Bloomberg L.P. Bloomberg terminal outage post-mortem report, 2015.
- [6] J. Boukhobza, S. Byna, Y.-H. Chang, S. Ibrahim, S. Jamil, Y. Kim, C.-L. Tsai, M. Zheng, and A. C. Zhou. A survey on the use of ai/ml for storage system design and optimization. *hal.science <hal-05528520>*, 2026.
- [7] J. Cao, O. R. Gatla, M. Zheng, D. Dai, V. Eswarappa, Y. Mu, and Y. Chen. Pfault: A general framework for analyzing the reliability of high-performance parallel file systems. In *Proceedings of the 2018 International Conference on Supercomputing*, pages 1–11, 2018.
- [8] CAST Software. Annual application software quality report, 2022.
- [9] CEA Tech. Frama-c: A software analysis toolset. <https://frama-c.com>, 2024. Version 28.
- [10] M. Chen et al. Evaluating large language models trained on code, 2021. arXiv:2107.03374.

- [11] A. Chou et al. An empirical study of operating systems errors. In *Proceedings of SOSP*, 2001.
- [12] D. Coleman, D. Ash, B. Lowther, and P. Oman. Using metrics to evaluate software system maintainability. *IEEE Computer*, 27(8):44–49, 1994.
- [13] E. Derr et al. Cfgnet: A network-based approach for tracking configuration dependencies. In *Proceedings of SANER*, 2022.
- [14] A. Fan et al. Large language models for software engineering: A survey. *IEEE Transactions on Software Engineering*, 2024.
- [15] M. Fowler. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 1999.
- [16] O. R. Gatla, D. Zhang, W. Xu, and M. Zheng. Understanding persistent-memory-related issues in the linux kernel. *ACM Transactions on Storage*, 19(4):1–28, 2023.
- [17] O. R. Gatla and M. Zheng. Understanding the fault resilience of file system checkers. In *9th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage 17)*, 2017.
- [18] O. R. Gatla, M. Zheng, M. Hameed, V. Dubeyko, A. Manzanares, F. Blagojevic, C. Guyot, and R. Mateescu. Towards robust file system checkers. *ACM Transactions on Storage (TOS)*, 14(4):1–25, 2018.
- [19] GitHub Inc. Octoverse 2024: Code duplication trends. <https://octoverse.github.com>, 2024.
- [20] Google DeepMind. Gemini 3.1 technical report, 2025.
- [21] M. H. Halstead. *Elements of Software Science*. Elsevier North-Holland, 1977.
- [22] R. Han, S. Byna, H. Tang, B. Dong, and M. Zheng. Prov-io: An i/o-centric provenance framework for scientific data on hpc systems. In *Proceedings of the 31st International*

Symposium on High-Performance Parallel and Distributed Computing, pages 213–226, 2022.

- [23] R. Han, O. R. Gatla, M. Zheng, J. Cao, D. Zhang, D. Dai, Y. Chen, and J. Cook. A study of failure recovery and logging of high-performance parallel file systems. *ACM Transactions on Storage (TOS)*, 18(2):1–44, 2022.
- [24] R. Han, C. Shi, T. Mahmud, Z. Yang, V. Esaulov, L. Wan, Y. Chen, J. Wayda, M. Wolf, and M. Zheng. Revisiting erasure codes: A configuration perspective. In *Proceedings of the 16th ACM Workshop on Hot Topics in Storage and File Systems*, pages 93–100, 2024.
- [25] R. Han, M. Zheng, S. Byna, H. Tang, B. Dong, D. Dai, Y. Chen, D. Kim, J. Hassoun, and D. Thorsley. Prov-io⁺: A cross-platform provenance framework for scientific data on hpc systems. *IEEE Transactions on Parallel and Distributed Systems*, 35(5):844–861, 2024.
- [26] B. Hui et al. Qwen2.5-coder technical report, 2024. arXiv:2409.12186.
- [27] JetBrains. State of developer ecosystem 2024. <https://www.jetbrains.com/lp/devecosystem-2024>, 2024.
- [28] C. E. Jimenez et al. Swe-bench: Can language models resolve real-world github issues?, 2023. arXiv:2310.06770.
- [29] R. Li et al. Starcoder: May the source be with you!, 2023. arXiv:2305.06161.
- [30] X. Lian, Y. Chen, R. Cheng, J. Huang, P. Thakkar, M. Zhang, and T. Xu. Large language models as configuration validators. <https://github.com/xlab-uiuc/ciri>, 2024. arXiv:2410.16918.
- [31] Linux Kernel Archive. Linux kernel statistics. <https://kernel.org>, 2025.
- [32] LLVM Project. Llvm slicer. <https://llvm.org>, 2023.

- [33] T. Mahmud, O. R. Gatla, D. Zhang, C. Love, R. Bumann, V. Girimaji, and M. Zheng. Analyzing configuration dependencies of file systems. *ACM Transactions on Computer Systems*, 43(4):1–28, 2025.
- [34] T. Mahmud, O. R. Gatla, D. Zhang, C. Love, R. Bumann, and M. Zheng. {ConfD}: Analyzing configuration dependencies of file systems for fun and profit. In *21st USENIX Conference on File and Storage Technologies (FAST 23)*, pages 199–214, 2023.
- [35] T. Mahmud, D. Zhang, O. R. Gatla, and M. Zheng. Understanding configuration dependencies of file systems. In *Proceedings of the 14th ACM Workshop on hot topics in storage and file systems*, pages 1–8, 2022.
- [36] R. C. Martin. *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall, 2003.
- [37] R. C. Martin. *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall, 2017.
- [38] T. J. McCabe. A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4):308–320, 1976.
- [39] Microsoft. Maintainability index range and meaning. <https://learn.microsoft.com>, 2007.
- [40] N. Nagappan, T. Ball, and A. Zeller. Mining metrics to predict component failures. In *Proceedings of ICSE*, pages 452–461, 2006.
- [41] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer. The impact of ai on developer productivity: Evidence from github copilot, 2023. arXiv:2302.06590.
- [42] Princeton NLP. Swe-bench verified. <https://www.swebench.com>, 2024.
- [43] A. Rabkin and R. Katz. Precomputing possible configuration error diagnoses. In *Proceedings of ASE*, 2011.

- [44] D. Riehle et al. Maintainability index revisited. In *Proceedings of XP*, 2019.
- [45] B. Roziere et al. Code llama: Open foundation models for code, 2023. arXiv:2308.12950.
- [46] D. Spadini. Radon: Python code metrics. <https://radon.readthedocs.io/>, 2020.
- [47] D. Spadini, M. Aniche, and A. Bacchelli. Pydriller: Python framework for mining software repositories. In *Proceedings of ESEC/FSE*, pages 571–574, 2018.
- [48] Stack Overflow. Developer survey 2024. <https://survey.stackoverflow.co/2024>, 2024.
- [49] SWE-bench. Swe-bench leaderboard. <https://www.swebench.com>, 2025.
- [50] M. Weiser. Program slicing. *IEEE Transactions on Software Engineering*, SE-10(4):352–357, 1984.
- [51] E. Xu, M. Zheng, F. Qin, Y. Xu, and J. Wu. Lessons and actions: What we learned from 10k {SSD-Related} storage system failures. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 961–976, 2019.
- [52] T. Xu et al. Do not blame users for misconfigurations. In *Proceedings of SOSP*, 2013.
- [53] D. Zhang, C. Egersdoerfer, T. Mahmud, M. Zheng, and D. Dai. Drill: Log-based anomaly detection for large-scale storage systems using source code analysis. In *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 189–199. IEEE, 2023.
- [54] D. Zhang, O. R. Gatla, W. Xu, and M. Zheng. A study of persistent memory bugs in the linux kernel. In *Proceedings of the 14th ACM International Conference on Systems and Storage*, pages 1–6, 2021.

- [55] D. Zhang and M. Zheng. Benchmarking for observability: The case of diagnosing storage failures. *BenchCouncil Transactions on Benchmarks, Standards and Evaluations*, 1(1):100006, 2021.
- [56] M. Zheng, V. T. Ravi, F. Qin, and G. Agrawal. GRace: a low-overhead mechanism for detecting data races in GPU programs. *ACM SIGPLAN Notices*, 46(8):135–146, 2011.
- [57] M. Zheng, V. T. Ravi, F. Qin, and G. Agrawal. GMRace: Detecting data races in GPU programs via a low-overhead scheme. *IEEE Transactions on Parallel and Distributed Systems*, 25(1):104–115, 2013.
- [58] M. Zheng, J. Tucek, D. Huang, F. Qin, M. Lillibridge, E. S. Yang, B. W. Zhao, and S. Singh. Torturing databases for fun and profit. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*, pages 449–464, 2014.
- [59] M. Zheng, J. Tucek, F. Qin, and M. Lillibridge. Understanding the robustness of {SSDs} under power fault. In *11th USENIX Conference on File and Storage Technologies (FAST 13)*, pages 271–284, 2013.
- [60] M. Zheng, J. Tucek, F. Qin, M. Lillibridge, B. W. Zhao, and E. S. Yang. Reliability analysis of ssds under power fault. *ACM Transactions on Computer Systems (TOCS)*, 34(4):1–28, 2016.
- [61] M. Zheng, D. Zhang, and A. Dajani. On fault tolerance of data storage systems: A holistic perspective. *Fault Tolerance in Modern Engineering Systems*, 2026.

CHAPTER A.

SLICELLM — SAMPLE FRAMA-C DEPENDENCY OUTPUT

The following is a representative excerpt of the Frama-C Program Dependence Graph output for the `juliandayFunc` function in SQLite's `date.c`. The output lists data dependencies (DD) and control dependencies (CD) for each statement in the function.

```

1  /* Statement: juliandayFunc body */
2  Data dependencies:
3    - p (arg 0) -> line 431: sqlite3_value_text(p)
4    - z -> line 432: parseIso8601Date(z, &x)
5    - x.iJD -> line 445: return sqlite3_result_double(context, r)
6
7  Control dependencies:
8    - if (z == 0) -> line 432-436
9    - if (x.isError) -> line 437-441
10
11 Registration:
12    - aDateTimeFuncs[12]: {"julianday", -1, 0, 0, juliandayFunc}

```

Listing A.1: Frama-C PDG output excerpt for `juliandayFunc`.

CHAPTER B.

CIRI EXT4 — SAMPLE INPUT CONFIGURATION

The following is the `ext4_ex.conf` sample input file used in CIRI ext4 experiments. Five misconfigurations are injected deliberately (annotated with comments for reference — comments are stripped before CIRI processing).

```
1  /* ext4 filesystem configuration */
2  ext4_journal_size = 999999;      /* MISCONFIG: exceeds
   total_blocks */
3  ext4_total_blocks = 1000;
4  ext4_encrypt = true;
5  ext4_encrypt_key_provider = none; /* MISCONFIG: key provider
   unset */
6  ext4_quota_enabled = true;
7  ext4_noquota_mode = true;        /* MISCONFIG: conflicts with
   quota */
8  ext4_auto_commit_interval = 0;   /* MISCONFIG: invalid with
   delayed alloc */
9  ext4_delayed_allocation = true;
10 ext4_hash_alg = teahash3;         /* Valid */
11 ext4_block_size = 4096;          /* Valid */
```

Listing B.1: ext4 sample configuration with injected misconfigurations.

CHAPTER C.

FINE-TUNING DATASET STATISTICS

Table C.1: SliceLLM Fine-Tuning Dataset Summary

Property	Value
Repository	SQLite (GitHub)
Extraction tool	pydriller 2.5
Total commits scanned	4,217
PRAGMA-related commits	89
Training examples	76
Validation examples	13
Base model	gpt-3.5-turbo-0125
Fine-tuned model ID	ft:gpt-3.5-turbo-0125:personal::BEPT4yFs
Training epochs	3